МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЧОРНОМОРСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ПЕТРА МОГИЛИ
Факультет комп'ютерних наук
Кафедра інтелектуальних інформаційних систем

ЗВІТ

до практичної роботи № 3
з дисципліни «Технології баз даних та знань»
на тему: «Робота зі СКБД SQLite. Виконання запитів на вибірку даних»

Виконав(ла): студент(ка) групи К26-1м
спеціальності Комп’ютерні науки
Климошенко Сергій

Перевірив(ла): _____

Оцінка: _____

Миколаїв – 2026

Тема та мета роботи

Тема: робота зі СКБД SQLite. Виконання запитів на вибірку даних.

Мета: навчитись засобами СКБД SQLite виконувати запити на вибірку даних.

Роботу виконано в середовищі macOS за допомогою консольного клієнта sqlite3 версії 3.51.0. Усі наведені нижче зображення є знімками реального сеансу роботи з інтерпретатором. Відмінності від методичних вказівок, зумовлені новішою версією СКБД (у методичці — 3.21.0): режим column автоматично підбирає ширину стовпців, тому назви на кшталт «Chicken/Fowl» не обрізаються; результати довгих запитів на знімках скорочено позначкою «[... вивід скорочено ...]». Ілюстративні схеми методички (загальна форма оператора select, конвеєр обробки, етапи виконання) подано в тексті, а не знімками консолі.

Хід виконання основного завдання

1. Тестова база даних

Надану студентом БД foods.db містить крім чотирьох таблиць предметної області (episodes, foods, food_types, foods_episodes) ще допоміжну порожню таблицю contacts, яка в завданні не використовується. Із чотирьох потрібних таблиць командою .dump сформовано дамп foods.sql, який відновлюється в нову БД food.db (рис. 1); далі переглядаємо перелік таблиць (рис. 2).

```
$ sqlite3 food.db < foods.sql
$ sqlite3 food.db
SQLite version 3.51.0 2025-06-12 13:14:41
Enter ".help" for usage hints.
sqlite>
```

Рис. 1. Відновлення БД із дампу foods.sql та запуск sqlite3

```
sqlite> .tables
episodes        food_types      foods           foods_episodes
sqlite>
```

Рис. 2. Перегляд переліку таблиць БД food

БД містить назви страв, що згадуються в епізодах серіалу «Seinfeld». Foods — головна таблиця: кожен запис є окремою стравою, назва зберігається в полі name, а поле type_id посилається на таблицю food_types (класи-

фікація: фаст-фуд, напої тощо). Таблиця foods_episodes пов'язує страви з епізодами (зв'язок «багато до багатьох»). Схему БД (даталогічну модель) наведено командою .schema (рис. 3).

```
sqlite> .schema
CREATE TABLE episodes (
  id integer primary key,
  season int,
  name text );
CREATE TABLE foods(
  id integer primary key,
  type_id integer,
  name text );
CREATE TABLE foods_episodes(
  food_id integer,
  episode_id integer );
CREATE TABLE food_types(
  id integer primary key,
  name text );
sqlite>
```

Рис. 3. Даталогічна модель БД «Food» (команда .schema)

2. Загальна форма оператора select

Найповніша форма оператора вибірки в SQLite має вигляд: select [distinct] heading from tables where predicate group by columns having predicate order by columns limit count,offset. Майже всі фрази необов'язкові. Оператор можна уявити як конвеєр: фраза from поєднує таблиці (join) в одне відношення R1; where відбирає записи (обмеження) та утворює R2; group by групує записи й застосовує агрегатні функції; having фільтрує групи; select виконує проекцію полів; order by упорядковує результат; limit обмежує кількість рядків.

3. Прості вибірки

Виконуємо просту вибірку зі списку типів страв (рис. 4) та проекцію — лише назви типів (рис. 5). Режим column та виведення заголовків увімкнено командами .mode column та .headers on.

```
sqlite> .headers on
sqlite> .mode column
sqlite> select * from food_types;
id  name
--  ------------
1   Bakery
2   Cereal
3   Chicken/Fowl
4   Condiments
5   Dairy
6   Dip
7   Drinks
8   Fruit
9   Junkfood
10  Meat
11  Rice/Pasta
12  Sandwiches
13  Seafood
14  Soup
15  Vegetables
sqlite>
```

Рис. 4. Вибірка зі списку типів страв

```
sqlite> select name from food_types;
name
------------
Bakery
Cereal
Chicken/Fowl
Condiments
Dairy
Dip
Drinks
Fruit
Junkfood
Meat
Rice/Pasta
Sandwiches
Seafood
Soup
Vegetables
sqlite>
```

Рис. 5. Назви типів страв

Вибираємо назви всіх серій серіалу (рис. 6). Кількість рядків велика, тому за допомогою limit обмежуємо результат п'ятьма серіями (рис. 7), а також п'ятьма першими стравами (рис. 8).

```
sqlite> select name from episodes;
name
----------------------------------
Good News Bad News
Male Unbonding
The Stake Out
The Robbery
The Stock Tip
The Ex-Girlfriend
The Pony Remark
[... вивід скорочено ...]
sqlite>
```

Рис. 6. Вибірка назв серій серіалу (вивід скорочено)

```
sqlite> select name from episodes LIMIT 5;
name
------------------
Good News Bad News
Male Unbonding
The Stake Out
The Robbery
The Stock Tip
sqlite>
```

Рис. 7. Обмеження результуючого списку фразою limit

```
sqlite> select * from foods limit 5;
id  type_id  name
--  -------  ----------------------
1   1        Bagels
2   1        Bagels, raisin
3   1        Bavarian Cream Pie
4   1        Bear Claws
5   1        Black and White cookies
sqlite>
```

Рис. 8. Перші 5 страв зі списку страв

Для зручності виводу змінюємо ширину стовпців командою .width (рис. 9).

```
sqlite> .width 3 3 30
sqlite> select * from foods limit 5;
id   typ  name
---  ---  ------------------------------
1    1    Bagels
2    1    Bagels, raisin
3    1    Bavarian Cream Pie
4    1    Bear Claws
5    1    Black and White cookies
sqlite>
```

Рис. 9. Вказання ширини стовпців при виводі даних

4. Фільтрування даних

Список серій другого сезону — рис. 10. Умови поєднуються булівськими операторами: страви 2-го та 3-го типу (рис. 11); ту ж вибірку реалізовано оператором in (рис. 12).

```
sqlite> .width auto
sqlite> select * from episodes where season = 2;
id  season  name
--  ------  --------------------
5   2       The Ex-Girlfriend
6   2       The Pony Remark
7   2       The Busboy
8   2       The Baby Shower
9   2       The Jacket
10  2       The Chinese Restaurant
11  2       The Phone Message
12  2       The Apartment
13  2       The Stranded
14  2       The Statue
15  2       The Heart Attack
16  2       The Revenge
17  2       The Deal
sqlite>
```

Рис. 10. Список серій 2-го сезону

```
sqlite> select * from foods where type_id = 2 or type_id = 3;
id  type_id  name
--  -------  ------------------------------------------
48  2        Bran
49  2        Cheerios
50  2        Corn Flakes
51  2        Double Crunch
52  2        Fruit Loops
53  2        Grape Nuts
54  2        Honey Combs
[... вивід скорочено ...]
sqlite>
```

Рис. 11. Список страв 2-го та 3-го типу (вивід скорочено)

```
sqlite> select * from foods where type_id in (2,3);
id  type_id  name
--  -------  ------------------------------------------
48  2        Bran
49  2        Cheerios
50  2        Corn Flakes
51  2        Double Crunch
52  2        Fruit Loops
53  2        Grape Nuts
54  2        Honey Combs
[... вивід скорочено ...]
sqlite>
```

Рис. 12. Використання оператора in (вивід скорочено)

Під час фільтрації за рядковим значенням застосовуємо шаблони оператора like: страви, назва яких пов'язана з морквою (рис. 13), та серії 3-го сезону зі словом «boyfriend» у назві (рис. 14).

```
sqlite> select * from foods where name like '%carrot%';
id   type_id  name
---  -------  -----------
8    1        Carrot Cake
363  14       Carrot Soup
387  15       Carrots
sqlite>
```

Рис. 13. Страви, назва яких пов'язана з морквою

```
sqlite> select * from episodes where season = 3 and name like '%boyfriend%';
id  season  name
--  ------  ---------------
32  3       The Boyfriend 1
33  3       The Boyfriend 2
sqlite>
```

Рис. 14. Серії 3-го сезону, в назві яких є слово «boyfriend»

5. Групування та агрегатні функції

Визначаємо загальну кількість серій (рис. 15), кількість серій 2-го сезону (рис. 16) та кількість сезонів із застосуванням count(distinct ...) (рис. 17). Зверніть увагу: у таблиці episodes є два службові записи без номера сезону (season = NULL: id 0 та id 180, спеціальний випуск), тому count(*) дає 181 запис, а кількість різних сезонів — 9.

```
sqlite> select count(*) from episodes;
count(*)
--------
181
sqlite>
```

Рис. 15. Загальна кількість серій у серіалі

```
sqlite> select count(*) from episodes where season = 2;
count(*)
--------
13
sqlite>
```

Рис. 16. Кількість серій 2-го сезону

```
sqlite> select count(distinct season) from episodes;
count(distinct season)
----------------------
9
sqlite>
```

Рис. 17. Кількість сезонів у серіалі

Речення group by розбиває вибірку на групи: підрахунок серій за сезонами (рис. 18); службові записи з NULL утворюють окрему групу. Результат агрегації фільтруємо реченням having — сезони, у яких більше 20 серій (рис. 19).

```
sqlite> select season, count(id)
   ...> from episodes
   ...> group by season;
season  count(id)
------  ---------
        2
1       4
2       13
3       22
4       24
5       22
6       24
7       24
8       22
9       24
sqlite>
```

Рис. 18. Підрахунок кількості серій за сезонами

```
sqlite> select season
   ...> from episodes
   ...> group by season
   ...> having count(id) > 20;
season
------
3
4
5
6
7
8
9
sqlite>
```

Рис. 19. Сезони, в яких більше 20 серій

6. Упорядкування виводу

Список типів страв за назвою (рис. 20); страви 1-го типу у зворотному порядку (рис. 21); серії за сезоном за зростанням і назвою за спаданням (рис. 22); сезони за кількістю серій за спаданням — упорядкування за результатом агрегатної функції (рис. 23).

```
sqlite> select name from food_types order by name;
name
------------
Bakery
Cereal
Chicken/Fowl
Condiments
Dairy
Dip
Drinks
Fruit
Junkfood
Meat
Rice/Pasta
Sandwiches
Seafood
Soup
Vegetables
sqlite>
```

Рис. 20. Список типів страв, упорядкований за назвою

```
sqlite> select * from foods where type_id = 1 order by name desc;
id   type_id  name
---  -------  --------------------------------
47   1        Wedding Cake (Royal)
46   1        Triscuits
45   1        Poppy Seed Muffins
35   1        Pizza Bagels
37   1        Pie (blueberry)
42   1        Pie (Strawberry)
41   1        Pie (Raspberry)
44   1        Pie (Peach)
40   1        Pie (Huckleberry)
[... вивід скорочено ...]
sqlite>
```

Рис. 21. Список страв 1-го типу, відсортований за назвою у зворотному порядку (вивід скорочено)

```
sqlite> select * from episodes order by season, name desc;
id   season  name
---  ------  --------------------------------
180          Reunion Special:The Seinfeld Story
0            Good News Bad News
4    1       The Stock Tip
2    1       The Stake Out
3    1       The Robbery
1    1       Male Unbonding
13   2       The Stranded
14   2       The Statue
16   2       The Revenge
[... вивід скорочено ...]
sqlite>
```

Рис. 22. Список серій, відсортований за сезоном (зростання) та назвою серії (спадання) (вивід скорочено)

```
sqlite> select season, count(id)
   ...> from episodes
   ...> group by season
   ...> order by 2 desc;
season  count(id)
------  ---------
9       24
7       24
6       24
4       24
8       22
5       22
3       22
2       13
1       4
        2
sqlite>
```

Рис. 23. Список сезонів, впорядкований за кількістю серій за спаданням

7. Запити до кількох таблиць

Для поєднання таблиць застосовуємо конструкцію join. Список страв із вказанням їхніх типів — рис. 24; салати з категорії овочі — рис. 25.

```
sqlite> select f.name, t.name
   ...> from foods f inner join food_types t
   ...> on f.type_id = t.id
   ...> limit 10;
name                      name
------------------------  ------
Bagels                    Bakery
Bagels, raisin            Bakery
Bavarian Cream Pie        Bakery
Bear Claws                Bakery
Black and White cookies   Bakery
Bread (with nuts)         Bakery
Butterfingers             Bakery
Carrot Cake               Bakery
Chips Ahoy Cookies        Bakery
Chocolate Bobka           Bakery
sqlite>
```

Рис. 24. Список страв із вказанням їхніх типів

```
sqlite> select f.name
   ...> from foods f inner join food_types t
   ...> on f.type_id = t.id
   ...> where f.name like '%salad%' and t.name = 'Vegetables';
name
------------
Big Salad
Chef Salad
Potato Salad
Salad
sqlite>
```

Рис. 25. Салати із категорії овочі

Приклади з поєднанням join та group by: перші 5 епізодів за кількістю згадувань про їжу (рис. 26); перші 3 сезони за кількістю згадувань різних

страв (рис. 27). У п'ятому рядку рейтингу епізодів у методичці стоїть «The Dinner Party», а в нашому виконанні — «The Wink»: обидва мають по 9 згадувань, тож порядок рівних значень не визначено, і він може відрізнятися залежно від версії СКБД.

```
sqlite> select e.name, count(f.food_id)
   ...> from episodes e inner join foods_episodes f
   ...> on e.id = f.episode_id
   ...> group by e.name
   ...> order by 2 desc
   ...> limit 5;
name            count(f.food_id)
--------------  ----------------
The Soup        23
The Fatigues    14
The Bubble Boy  12
The Finale 1    10
The Wink        9
sqlite>
```

Рис. 26. Перші 5 епізодів за кількістю згадувань про їжу

```
sqlite> select e.season, count(distinct f.food_id)
   ...> from episodes e inner join foods_episodes f
   ...> on e.id = f.episode_id
   ...> group by e.season
   ...> order by 2 desc
   ...> limit 3;
season  count(distinct f.food_id)
------  -------------------------
9       82
5       73
6       69
sqlite>
```

Рис. 27. Перші 3 сезони за кількістю згадувань різних страв

У скількох серіях кожного сезону згадується морозиво — рис. 28; у яких серіях 5-го сезону згадується випічка — рис. 29.

```
sqlite> select e.season, count(distinct e.id)
   ...> from episodes e inner join foods_episodes fe
   ...> on e.id = fe.episode_id
   ...> inner join foods f on fe.food_id = f.id
   ...> where f.name like '%Ice Cream%'
   ...> group by e.season;
season  count(distinct e.id)
------  --------------------
2       2
4       1
9       2
sqlite>
```

Рис. 28. У скількох серіях кожного із сезонів згадується про морозиво

```
sqlite> select distinct e.*
   ...> from episodes e inner join foods_episodes fe
   ...> on e.id = fe.episode_id
   ...> inner join foods f on fe.food_id = f.id
   ...> inner join food_types t on f.type_id = t.id
   ...> where t.name = 'Bakery' and e.season = 5;
id  season  name
--  ------  ----------------------
67  5       The Sniffing Accountant
82  5       The Raincoats 2
76  5       The Dinner Party
66  5       The Glasses
78  5       The Pie
75  5       The Stall
sqlite>
```

Рис. 29. У яких серіях 5-го сезону згадується випічка

Індивідуальне завдання

Для БД «Страви» (окрема копія food_ind.db, відновлена з того самого дампу) реалізовано сім запитів на вибірку.

1. Типи страв, що починаються з літери «D» (рис. 30). Оператор like у SQLite нечутливий до регістру для латинських літер.

```
sqlite> .headers on
sqlite> .mode column
sqlite> select * from food_types where name like 'D%';
id  name
--  ------
5   Dairy
6   Dip
7   Drinks
sqlite>
```

Рис. 30. Типи страв, що починаються з літери «D»

2. Страви з кодом категорії від 3 до 5 — оператор between (рис. 31).

```
sqlite> select * from foods where type_id between 3 and 5;
id   type_id  name
---  -------  ------------------------------------------
63   3        Broiled Chicken
64   3        Casserole
65   3        Chicken
66   3        Chicken (for wedding)
67   3        Chicken Cashew (not ordered)
68   3        Chicken Kiev
69   3        Kung-Pao Chicken
70   3        Chicken Marsala
71   3        Chicken Piccata
[... вивід скорочено ...]
sqlite>
```

Рис. 31. Страви із кодом категорії від 3 до 5 (вивід скорочено)

3. Серії 9 сезону, в назві яких зустрічається «fin» (рис. 32). Крім очікуваних «The Finale 1» та «The Finale 2» вибірка містить «The Merv Griffin Show», оскільки «Griffin» містить підрядок «fin».

```
sqlite> select * from episodes where season = 9 and name like '%fin%';
id   season  name
---  ------  --------------------
161  9       The Merv Griffin Show
178  9       The Finale 1
179  9       The Finale 2
sqlite>
```

Рис. 32. Серії 9 сезону, в назві яких є «fin»

4. Типи страв із кількістю страв у кожному (рис. 33); використано left join, щоб у результат потрапили й типи без страв (таких немає).

```
sqlite> select t.name, count(f.id) as foods_count
   ...> from food_types t left join foods f on f.type_id = t.id
   ...> group by t.id, t.name;
name          foods_count
------------  -----------
Bakery        48
Cereal        15
Chicken/Fowl  23
Condiments    22
Dairy         17
Dip           4
Drinks        60
Fruit         23
Junkfood      61
Meat          36
Rice/Pasta    16
Sandwiches    23
Seafood       14
Soup          19
Vegetables    32
sqlite>
```

Рис. 33. Типи страв із кількістю страв, що входять до них

5. Перші 3 типи страв за кількістю страв (рис. 34).

```
sqlite> select t.name, count(f.id) as foods_count
   ...> from food_types t inner join foods f on f.type_id = t.id
   ...> group by t.id, t.name
   ...> order by foods_count desc
   ...> limit 3;
name      foods_count
--------  -----------
Junkfood  61
Drinks    60
Bakery    48
sqlite>
```

Рис. 34. Перші 3 типи страв за кількістю страв

6. Страва, що згадується у найбільшій кількості епізодів (рис. 35). Найближчий конкурент має 4 епізоди, тож лідер єдиний.

```
sqlite> select f.name, count(distinct fe.episode_id) as episodes_count
   ...> from foods f inner join foods_episodes fe on fe.food_id = f.id
   ...> group by f.id, f.name
   ...> order by episodes_count desc
   ...> limit 1;
name     episodes_count
-------  --------------
Hot Dog  5
sqlite>
```

Рис. 35. Страва, що згадується у найбільшій кількості епізодів

7. Кількість різних типів страв, що згадуються в кожному із сезонів (рис. 36); порожнє значення сезону відповідає службовим записам без номера сезону.

```
sqlite> select e.season, count(distinct f.type_id) as types_count
   ...> from episodes e inner join foods_episodes fe on fe.episode_id = e.id
   ...> inner join foods f on f.id = fe.food_id
   ...> group by e.season
   ...> order by e.season;
season  types_count
------  ------------
        2
1       6
2       10
3       11
4       14
5       13
6       15
7       14
8       13
9       15
sqlite>
```

Рис. 36. Кількість різних типів страв у кожному із сезонів

Висновки

У ході виконання практичної роботи було опановано засоби СКБД SQLite для виконання запитів на вибірку даних.

— Оператор select працює як конвеєр: from (з'єднання) → where (обмеження) → group by / having (групування й фільтрація груп) → select (проекція) → order by → limit; всі фрази, крім select, необов'язкові.

— Фільтрування виконується предикатами з оператором in, like, between та булівськими операторами; like нечутливий до регістру для ASCII, що слід враховувати (наприклад, «fin» знаходить і «Griffin»).

— Агрегатні функції count, count(distinct ...) разом із group by і having дозволяють підраховувати показники по групах; order by допускає упорядкування за кількома полями та за результатом агрегації.

— Для запитів до кількох таблиць застосовується join; поєднання join із group by дозволяє отримувати рейтинги (епізоди, сезони, типи страв).

— У даних є особливості, які впливають на результат: два епізоди без номера сезону (NULL) утворюють окрему групу, а за рівних значень агрегата порядок рядків не визначений.

Основне та індивідуальне завдання виконано в повному обсязі, усі запити виконано в реальному sqlite3 версії 3.51.0.

Список використаних джерел

1. Кренке Д. Теория и практика построения баз данных. 8-е изд. — СПб. : Питер, 2003. — 800 с.

2. Дейт К. Дж. Введение в системы баз данных. 7-е издание : пер. с англ. — М. : Издательский дом «Вильямс», 2001. — 1072 с.

3. Грофф Дж., Вайнберг П. SQL : Повне керівництво : пер. з англ. — 2-ге вид. — К. : Видавнича група BHV, 2001. — 816 с.

4. Конноллі Т., Бегг К. Базы данных. Проектирование, реализация и сопровождение. Теория и практика. 3-е издание : пер. с англ. — М. : Издательский дом «Вильямс», 2003. — 1440 с.

5. Фісун М. Т., Ніколенко С. Г. Створення та ведення баз даних засобами мови Jet SQL : методичні вказівки. — Миколаїв : вид-во ЧДУ ім. Петра Могили, 2009. — 83 с.

6. SQLite Documentation. SELECT [Електронний ресурс]. — Режим доступу: https://www.sqlite.org/lang_select.html

7. SQLite Documentation. Aggregate Functions [Електронний ресурс]. — Режим доступу: https://www.sqlite.org/lang_aggfunc.html

8. SQLite Documentation. Command Line Shell For SQLite [Електронний ресурс]. — Режим доступу: https://www.sqlite.org/cli.html